

How control expression is generated in ModuleFinder

SRC/ControlDataGenerator.py — step-by-step derivation

1. Setup and notation

Symbol	Shape	Meaning
G	scalar	number of genes
P	scalar	number of programs (default 15, derived from GOProgramGenerator)
N	scalar	number of control cells to generate
W	$G \times P$	gene-program coupling matrix (built by GeneRegulatoryNetwork)
ρ	$P \times P$	program-activity correlation matrix
$\mathbf{b}$	$G \times 1$	per-gene baseline log-expression
r	$G \times 1$	per-gene NB dispersion
$\mathbf{h}_i$	$P \times 1$	latent (Gaussian) program-activity vector for cell i
$\mathbf{a}_i$	$P \times 1$	non-negative program-activity vector for cell i
$\mathbf{e}_i$	$G \times 1$	log-scale expression of cell i
λ_i	$G \times 1$	expected counts of cell i
$\mathbf{x}_i$	$G \times 1$	sampled counts of cell i

The function chain in **ControlDataGenerator**:

`sample_program_activities()` → `generate_expression()` → `expression_to_counts()`

2. Step 1 — Program-correlation matrix

A $P \times P$ symmetric matrix ρ encoding biologically motivated dependencies between programs. The function `_define_program_correlations()` initialises an identity matrix, then for each pair of programs in a hard-coded list samples

$$\rho_{p,q} \sim \mathcal{N}(\bar{\rho}_{p,q}, \sigma^2), \quad \sigma = 0.05,$$

clipped to $[-1, 1]$, with means $\bar{\rho}_{p,q}$ given by:

Pair	$\bar{\rho}$
cell_cycle metabolism	+0.5
cell_cycle ribosome_biogenesis	+0.6
apoptosis cell_cycle	-0.4
oxidative_stress er_stress	+0.5
oxidative_stress autophagy	+0.4
er_stress autophagy	+0.4

Pair	$\bar{\rho}$
hypoxia_response angiogenesis	+0.6
hypoxia_response metabolism	+0.3
immune_response inflammatory_response	+0.7
emt cell_adhesion	-0.5
dna_repair cell_cycle	+0.3

Pairs not listed stay at 0. The resulting matrix is then projected to its **nearest positive-definite matrix** (`_nearest_positive_definite`) so it's a valid covariance kernel:

$$\rho \leftarrow \text{nearestPSD}\left(\frac{\rho + \rho^\top}{2}\right).$$

3. Step 2 — Per-cell program activities

For each cell $i \in \{1, \dots, N\}$, sample a *latent* program-activity vector from a multivariate Gaussian:

$$\mathbf{h}_i \sim \mathcal{N}(\mu, \sigma_a^2 \rho)$$

where $\mu \in \mathbb{R}^P$ is the mean activity vector (scalar broadcast or per-program; `activity_mean` arg) and σ_a is the activity standard deviation (`activity_std`, default 1.0). When `use_correlations=False` the off-diagonal of ρ is dropped: $\mathbf{h}_i \sim \mathcal{N}(\mu, \sigma_a^2 I_P)$.

Pass the latent through a softplus so activities are non-negative:

$$\mathbf{a}_i = \log(1 + e^{\mathbf{h}_i})$$

Per-cell activities $\mathbf{a}_i$ are stacked into the $N \times P$ matrix A .

The softplus has slope ~ 1 in the regime $\mathbf{h} \gg 0$ (so high-activity programs are nearly the latent value) and saturates to 0 for $\mathbf{h} \ll 0$ — that's where the “off” programs sit.

4. Step 3 — Log-expression from program activities

For each cell i :

$$\mathbf{e}_i = W \mathbf{a}_i + \mathbf{b} + \eta_i + \log s_i \cdot \mathbf{1}_G$$

component-wise,

$$e_{i,g} = \sum_{p=1}^P W_{g,p} a_{i,p} + b_g + \eta_{i,g} + \log s_i.$$

The four terms:

1. **Programmatic component** $W \mathbf{a}_i$. Maps program activities to per-gene expression contributions through the GRN's W matrix. Genes with $W_{g,p} > 0$ are co-active with program p ; $W_{g,p} < 0$ means anti-active. Genes outside any program have $W_{g,\cdot} \equiv 0$ and only see baseline + noise.
2. **Per-gene baseline** $\mathbf{b}$. Sampled once at construction time:

$$b_g \sim \mathcal{N}(0.5, 2.0^2),$$

in *natural log* space. The heavy tail spans roughly four to five orders of magnitude in linear space, matching real scRNA-seq.

3. **Biological noise** η_i . Per-cell, per-gene Gaussian:

$$\eta_{i,g} \stackrel{\text{iid}}{\sim} \mathcal{N}(0, 0.2^2).$$

4. **Cell size factor** s_i . Per-cell scalar:

$$s_i \sim \text{Lognormal}(0, c_s^2)$$

with `cell_size_factor_cv = 0.3` by default. Adds the same shift $\log s_i$ to every gene in cell i — modelling library-size variability.

5. Step 4 — Log-expression to counts

Convert log-scale expression to expected counts and re-scale every cell to a target library size T (default `total_counts_per_cell = 10{,}000`):

$$\lambda_{i,g} = e^{e_{i,g}}, \quad \tilde{\lambda}_{i,g} = \lambda_{i,g} \cdot \frac{T}{\sum_{g'} \lambda_{i,g'}}.$$

Sample integer counts from one of two distributions per cell-gene cell:

Negative binomial (default):

$$x_{i,g} \sim \text{NB}(r_g, p_{i,g}), \quad p_{i,g} = \frac{r_g}{r_g + \tilde{\lambda}_{i,g}}$$

with per-gene dispersion $r_g \sim \text{Uniform}(0.1, 2.0)$ drawn once at construction (`dispersion_params`). Mean = $\tilde{\lambda}_{i,g}$, variance = $\tilde{\lambda}_{i,g} + \tilde{\lambda}_{i,g}^2/r_g$ — Poisson-like at low $\tilde{\lambda}$, overdispersed at high $\tilde{\lambda}$.

Poisson (set `count_distribution='poisson'`):

$$x_{i,g} \sim \text{Poisson}(\tilde{\lambda}_{i,g}).$$

Output: AnnData with $\mathbf{X} = N \times G$ integer count matrix, `obs_names = ["Cell_0", "Cell_1", ...]`, `var_names` = gene symbols from the GRN.

6. Defaults at a glance

Parameter	Default	Where set
<code>n_cells</code>	required arg	<code>generate_control_data(n_cells=...)</code>
<code>activity_mean</code>	0.0 (scalar broadcast)	scalar or per-program vector
<code>activity_std</code> (σ_a)	1.0	latent activity stddev
<code>use_correlations</code>	True	apply ρ vs identity
baseline mean	0.5	$b_g \sim \text{Normal mean}$
baseline std	2.0	$b_g \sim \text{Normal std (heavy-tailed)}$
biological-noise std	0.2	$\eta_{i,g}$
<code>cell_size_factor_cv</code>	0.3	log-normal cell-size
<code>total_counts_per_cell</code> (T)	10{,}000	row library-size target
dispersion range r_g	U(0.1, 2.0)	once per gene
<code>count_distribution</code>	negative_binomial	NB or Poisson
seed	42	reproducibility

7. Notes and design choices

- **Program activity is correlated by construction.** This is what gives rise to cell-state-like structure in PCA/UMAP of the controls — the latent space has P components, but the activities are sampled from a multivariate normal whose correlation pattern reflects shared biology.

- **Softplus is non-linear and asymmetric.** The Gaussian latent has full support, but $\mathbf{a}_i$ is non-negative. Programs whose latent mean is large positive behave nearly linearly; programs with negative latent mean stay close to zero (and so contribute little to $W\mathbf{a}_i$). This is how the simulator implements “context-specific control populations” — by setting a per-program μ that is positive for some programs and negative for others.
- **Library size lives in log-space and is then re-normalised.** The $\log s_i$ shift in step 3 only adds variance to row sums; the explicit per-row re-scaling to T in step 4 fixes the library size to a target. Together they create cell-to-cell variability in *relative* gene proportions while keeping the total counts comparable.
- **expression_to_counts is a double Python loop** over cells $\times$ genes for NB sampling. Vectorisable but currently fine for $N \cdot G \lesssim 10^7$.
- **Per-gene dispersion is sampled once at construction**, not estimated from data — this is a synthetic prior on count-level overdispersion. If you want to match a real dataset’s dispersion, override `dispersion_params` after construction.
- **The downstream simulator** (`PerturbSeqSimulator`) takes whatever `AnnData` this generator produces (or any real control `AnnData`) as its `control_adata` argument and **re-estimates** per-gene dispersion from those counts via method-of-moments. So the “control dispersion” used during KO simulation is the empirical one, not the synthetic `dispersion_params` here.